COS 350 Lecture 12

unix: ln

directory structure

inodes

unix: mkdir, rmdir,

unlink, link, symlink

rename, chdir

case study: pwd

NAME

ln - make links between files

SYNOPSIS

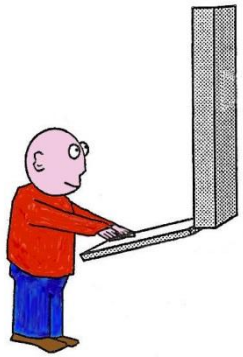
ln [options] existing-file new-file
ln [options] existing-files [directory]

DESCRIPTION

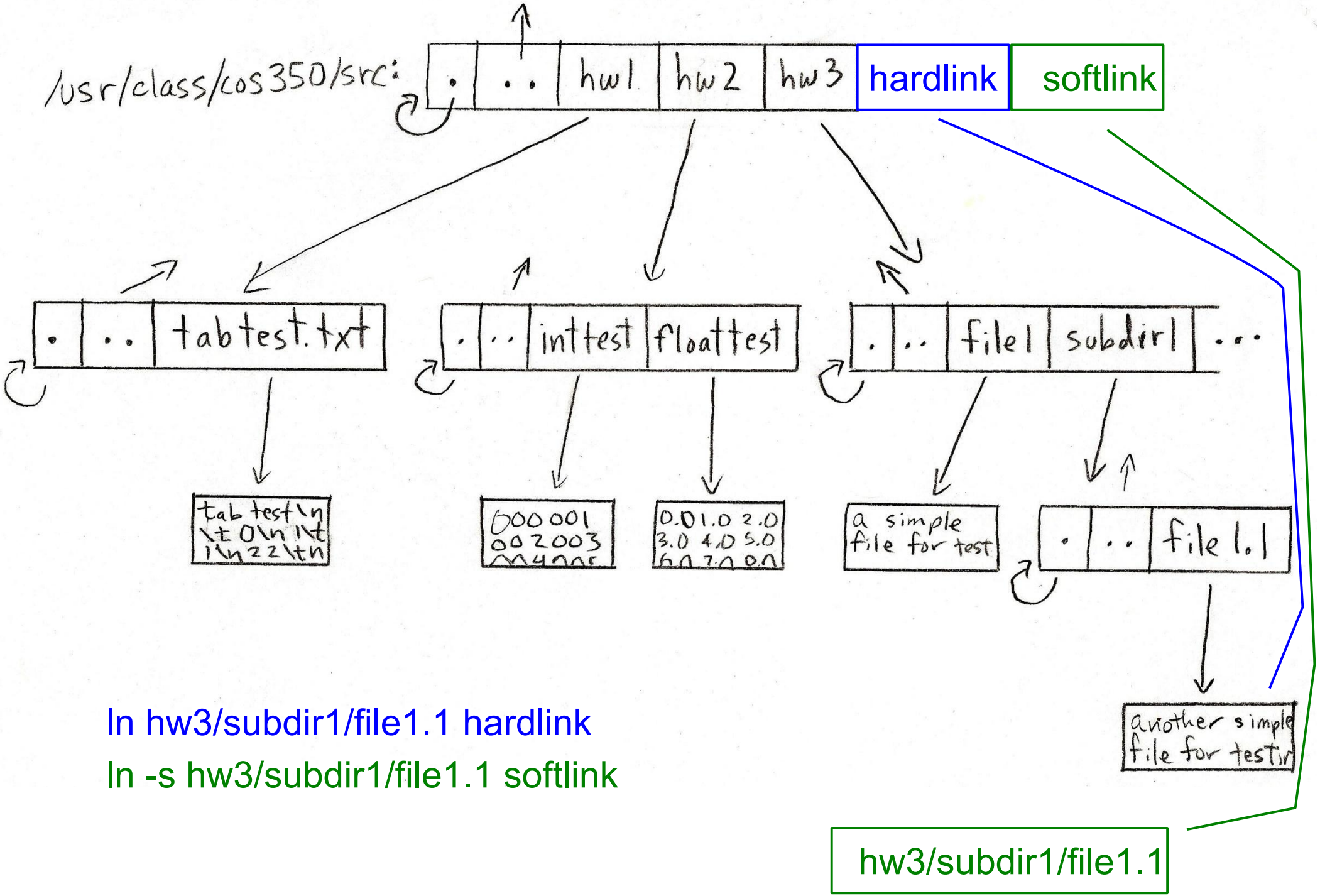
In the 1st form, create a link to 'existing-file' with the name 'new-file'
In the 2nd form, creates links to the 'existing-files' in 'directory' and uses the same names as the original files.

OPTIONS

-s make symbolic links instead of hard links
only symbolic links can be used for directories or cross into a different device (disk drive)



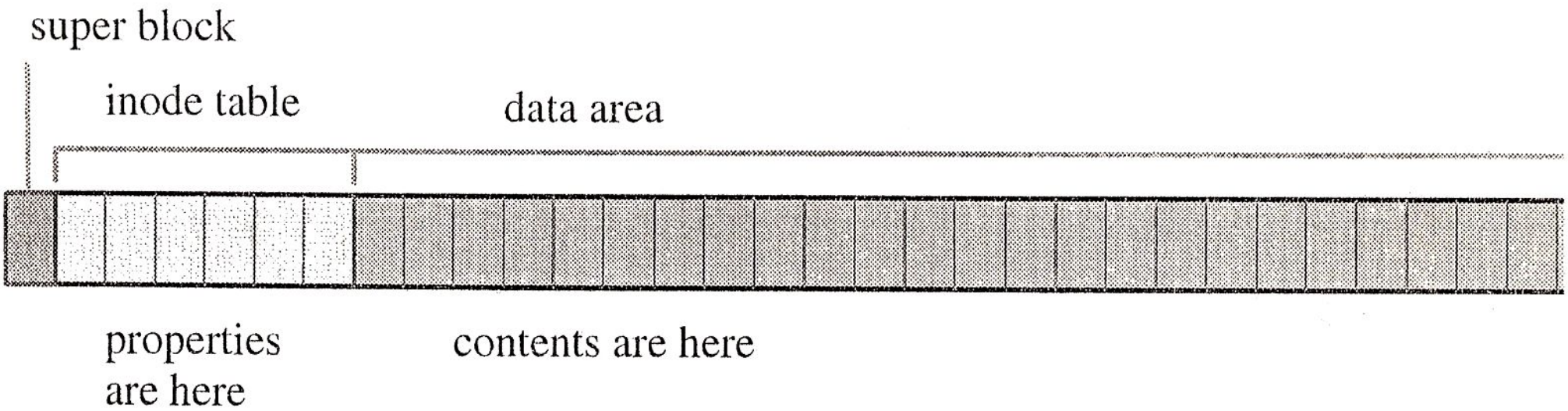
```
ln file1 file2
ls -l

ln /usr/class/cos350/prog2/shake.txt shake.txt

ln -s /usr/class/cos350/prog2/shake.txt shake.txt
ls -l
```



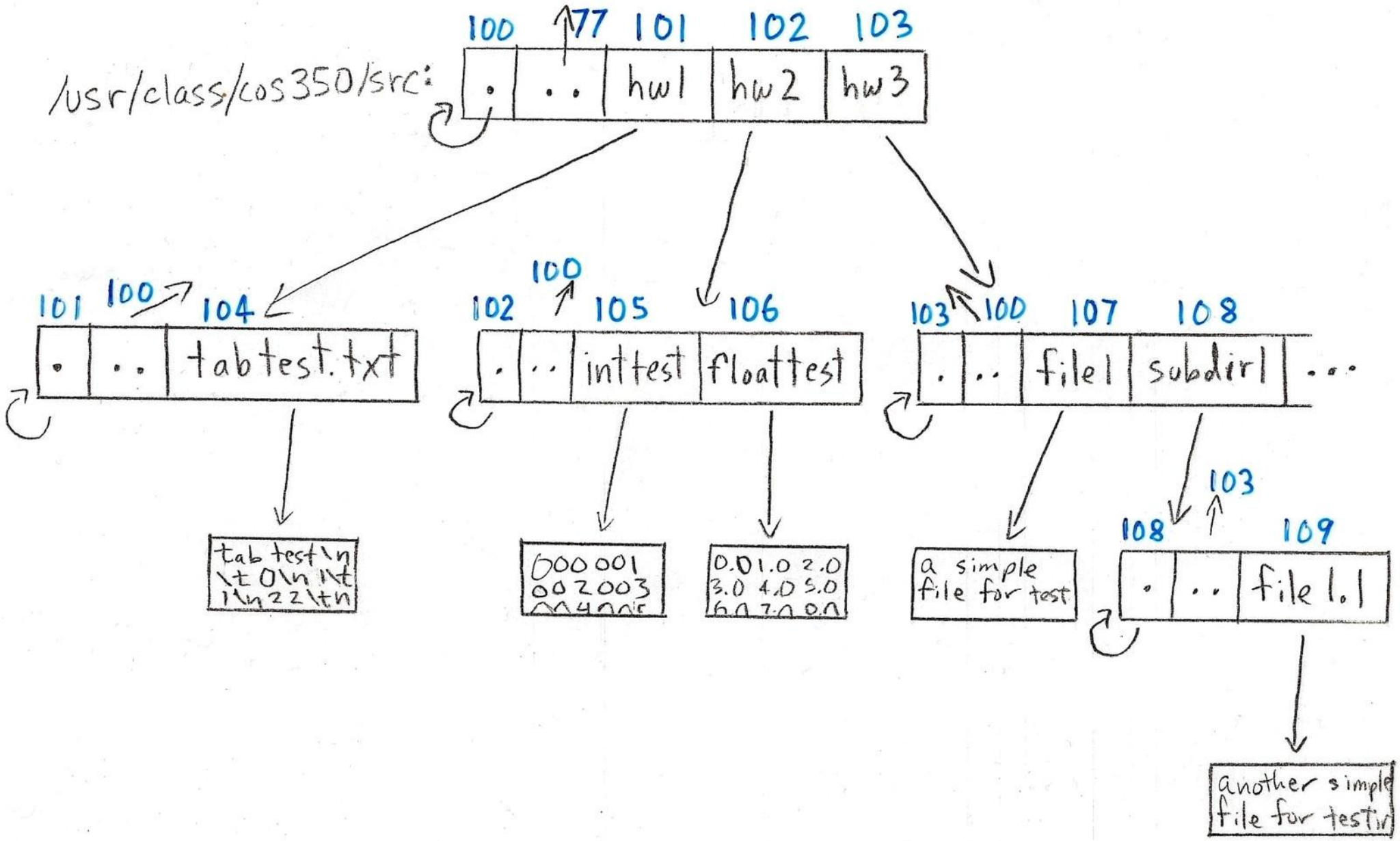
ln hw3/subdir1/file1.1 hardlink

ln -s hw3/subdir1/file1.1 softlink

hw3/subdir1/file1.1



The three regions of a file system.


/usr/class/cos350/src:
100
77
101
102
103
.
..
hw1
hw2
hw3
101
100
104
.
..
tabtest.txt
tab test\n
\t 0\n 1\t
1\n 2 2\t\n
102
100
105
106
.
..
inttest
floattest
000001
002003
0.0 1.0 2.0
3.0 4.0 5.0
103
100
107
108
.
..
file1
subdir1
...
a simple
file for test
108
103
109
.
..
file1.1
another simple
file for testin

NAME

mkdir - create a directory

SYNOPSIS

```
#include <sys/stat.h>
#include <sys/types.h>

int mkdir(const char *pathname, mode_t mode);
```

DESCRIPTION

mkdir() attempts to create a directory named pathname.

The argument mode specifies the permissions to use. It is modified by the process's umask in the usual way: the permissions of the created directory are (mode & ~umask & 0777).

NAME

rmdir - delete a directory

SYNOPSIS

```
#include <unistd.h>

int rmdir(const char *pathname);
```

DESCRIPTION

rmdir() deletes a directory, which must be empty.

RETURN VALUE

On success, zero is returned. On error, -1 is returned, and errno is set appropriately.

NAME

unlink - delete a name and possibly the file it refers to

SYNOPSIS

```
#include <unistd.h>

int unlink(const char *pathname);
```

DESCRIPTION

unlink() deletes a name from the file system. If that name was the last link to a file and no processes have the file open the file is deleted and the space it was using is made available for reuse.

If the name was the last link to a file but any processes still have the file open the file will remain in existence until the last file descriptor referring to it is closed.

If the name referred to a symbolic link the link is removed.

NAME

link - make a new name for a file

SYNOPSIS

```
#include <unistd.h>

int link(const char *oldpath, const char *newpath);
```

DESCRIPTION

link() creates a new link (also known as a hard link) to an existing file.

If newpath exists it will not be overwritten.

This new name may be used exactly as the old one for any operation; both names refer to the same file (and so have the same permissions and ownership) and it is impossible to tell which name was the "original".

NAME

symlink - make a new name for a file

SYNOPSIS

```
#include <unistd.h>

int symlink(const char *oldpath, const char *newpath);
```

DESCRIPTION

symlink() creates a symbolic link named newpath which contains the string oldpath.

Symbolic links are interpreted at run time as if the contents of the link had been substituted into the path being followed to find a file or directory.

Symbolic links may contain .. path components, which (if used at the start of the link) refer to the parent directories of that in which the link resides.

A symbolic link (also known as a soft link) may point to an existing file or to a nonexistent one; the latter case is known as a dangling link.

NAME

rename - change the name or location of a file

SYNOPSIS

```
#include <stdio.h>

int rename(const char *oldpath, const char *newpath);
```

DESCRIPTION

rename() renames a file, moving it between directories if required. Any other hard links to the file (as created using link(2)) are unaffected. Open file descriptors for oldpath are also unaffected.

If newpath already exists it will be atomically replaced (subject to a few conditions; see ERRORS below), so that there is no point at which another process attempting to access newpath will find it missing.

CHDIR(2) Linux Programmer's Manual CHDIR(2)

NAME

chdir - change working directory

SYNOPSIS

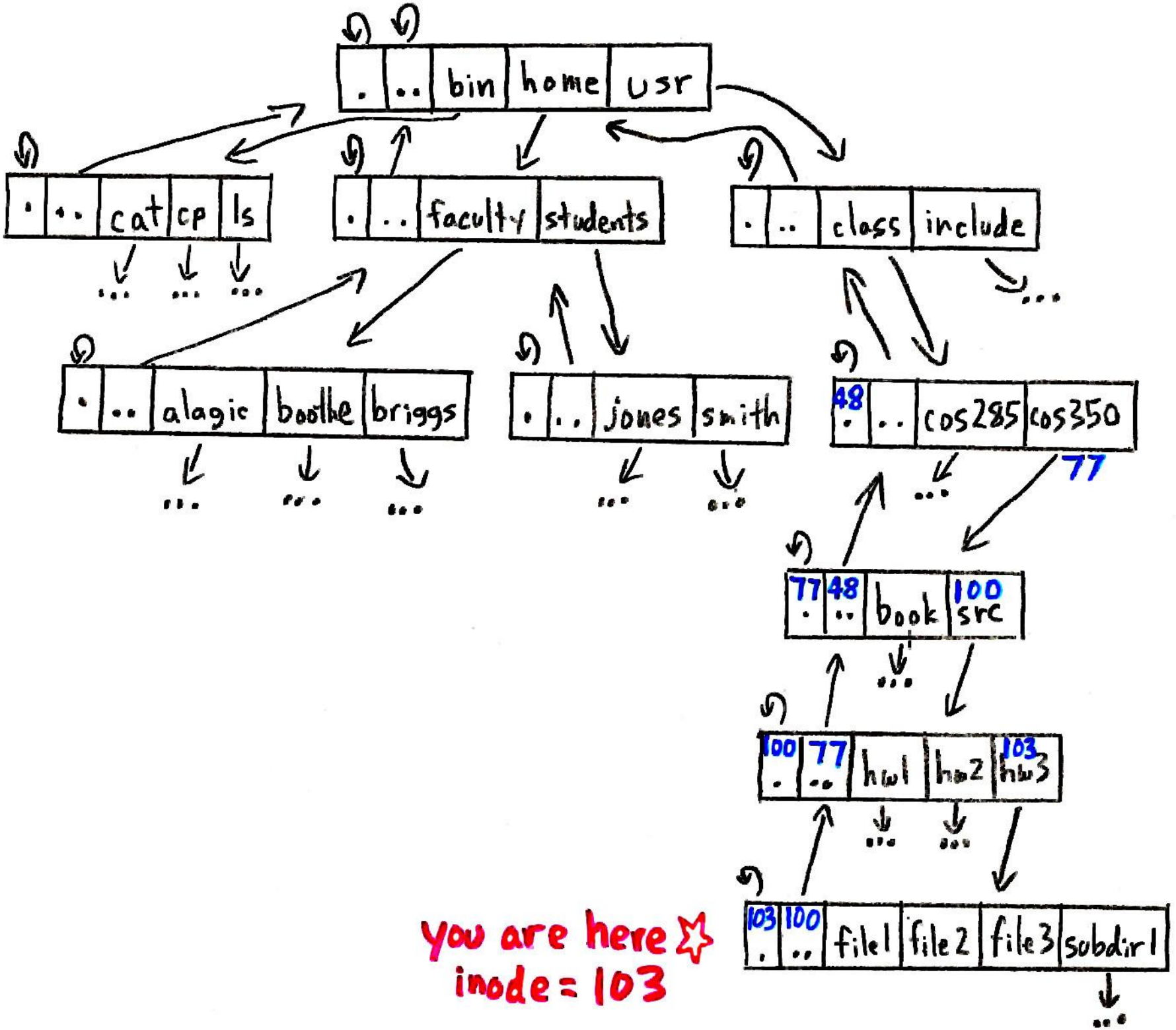
```
#include <unistd.h>

int chdir(const char *path);
```

DESCRIPTION

chdir() changes the current working directory of the calling process to the directory specified in path.

case study: pwd

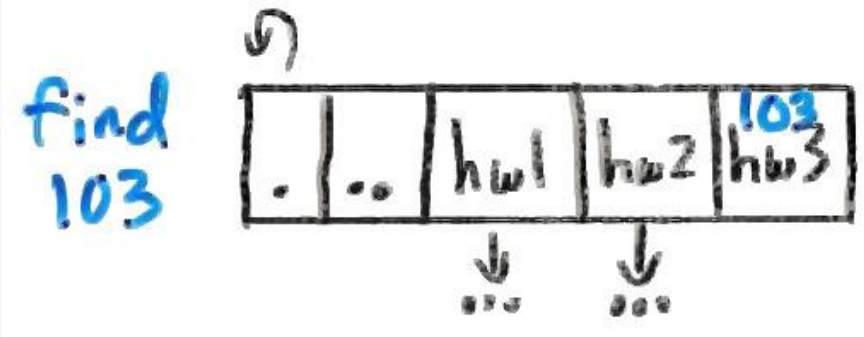
```
unix> pwd
/usr/class/cos350/src/hw3
```


. .. bin home usr
. .. cat cp ls
. .. faculty students
. .. class include
. .. alagic boothe briggs
. .. jones smith
48 . .. cos285 cos350
77
77 . 48 .. book 100 src
100 . 77 .. hw1 hw2 103 hw3
103 . 100 .. file1 file2 file3 subdir1
you are here
inode = 103

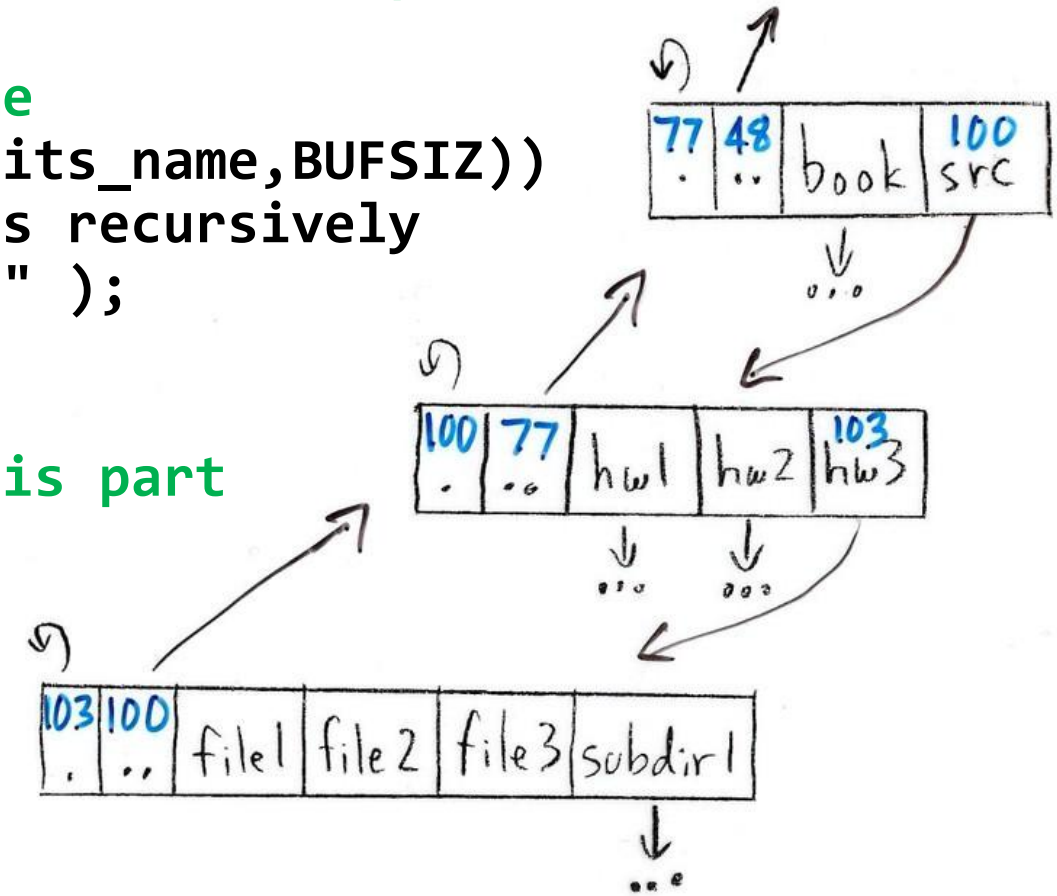
```c
/* pwd1.c: a simplified version of pwd
 *
 * starts in current directory and recursively
 * climbs up to root of filesystem, prints top part
 * then prints current part
 *
 * uses readdir() to get info about each thing
 *
 * bug: prints an empty string if run from "/"
 */
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <dirent.h>

ino_t   get_inode(char *);
void    printpathto(ino_t);
int     inum_to_name(ino_t , char *, int );

int main() {
  printpathto( get_inode( "." ) );  /* print path to here */
  putchar('\n');                    /* then add newline */
  return 0;
}
```

```c
/*
 *      returns inode number of the file
 */
ino_t get_inode( char *fname ) {
  struct stat info;

  if ( stat( fname , &info ) == -1 ){
    fprintf(stderr, "Cannot stat ");
    perror(fname);
    exit(1);
  }

  return info.st_ino;
}
```

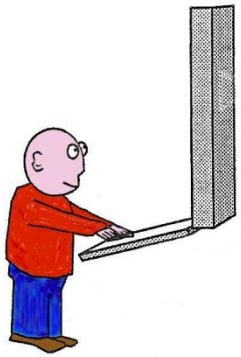
```
// looks through current directory for a file with this inode
// copies its name into namebuf
// returns 0 if couldn't find it, 1 if OK
int inum_to_name(ino_t inode, char *namebuf, int buflen) {
  DIR           *dir_ptr;                /* the directory */
  struct dirent *direntp;                /* each entry    */

  dir_ptr = opendir( "." );
  if ( dir_ptr == NULL ){
    perror( "." );
    exit(1);
  }

  //  search directory for a file with specified inum
  while ( ( direntp = readdir( dir_ptr ) ) != NULL )
    if ( direntp->d_ino == inode )
      {
        strncpy( namebuf, direntp->d_name, buflen);
        namebuf[buflen-1] = '\0';   /* just in case */
        closedir( dir_ptr );
        return 1;
      }
  strcpy(namebuf, "???");   // couldn't find it
  return 0;
}
```



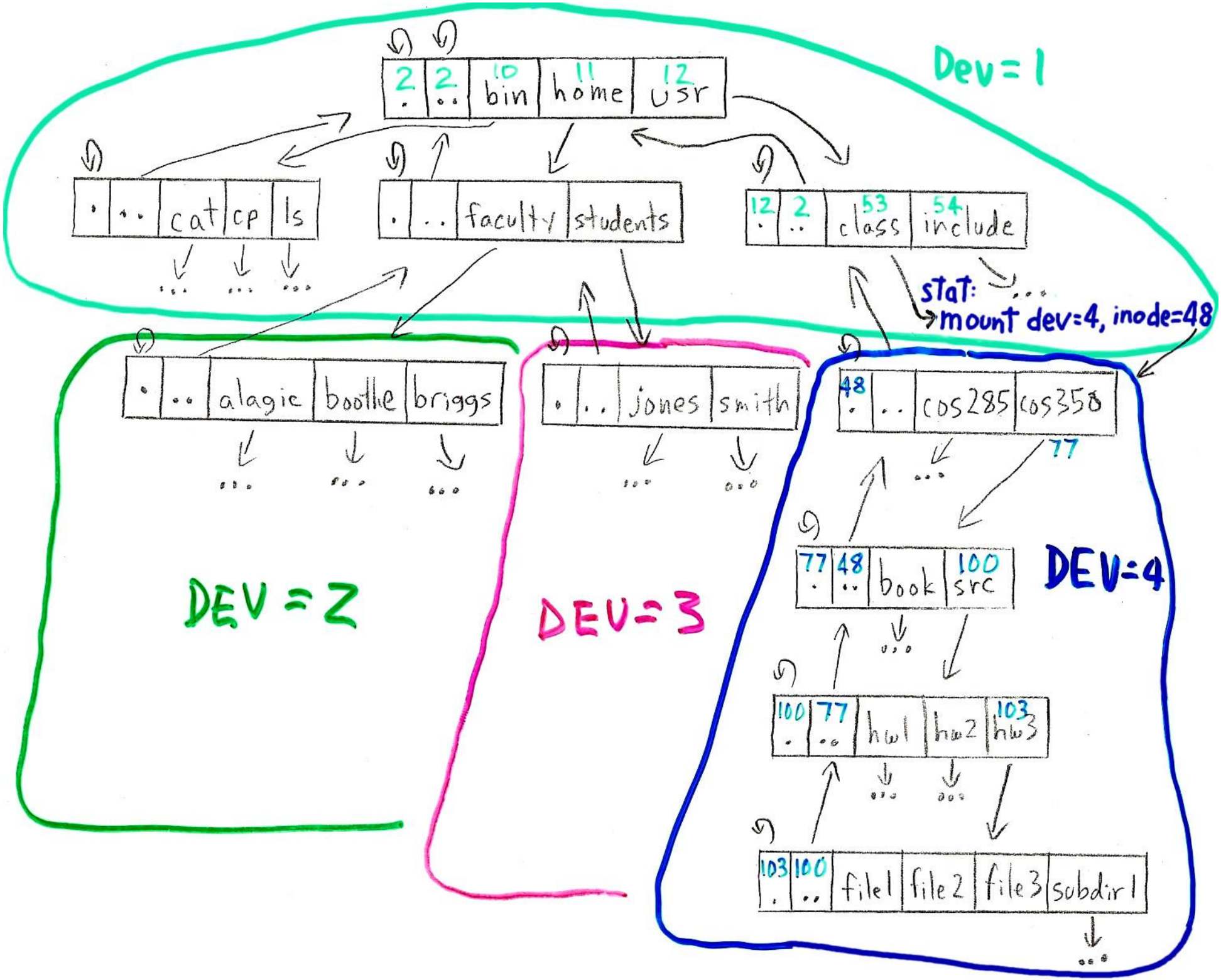
```c
// prints path leading down to an object with this inode
// recursively prints in reverse order of discovery
void printpathto( ino_t this_inode ) {
  ino_t my_inode ;
  char  its_name[BUFSIZ];

  if ( get_inode("..") == this_inode ) //root points to self
    return;

  chdir( ".." );                          // up one dir

  // find this dirs actual name
  if (inum_to_name(this_inode,its_name,BUFSIZ))
    { // 1st: print parent dirs recursively
      my_inode = get_inode( "." );
      printpathto( my_inode );
    }
  // 2nd: now print name of this part
  printf("/%s", its_name );
}
```



this_inode = 103



pwd1


Dev=1
2 . 2 .. 10 bin 11 home 12 usr
. .. cat cp ls
. .. faculty students
12 . 2 .. 53 class 54 include
stat:
mount dev=4, inode=48
. .. alagic boothe briggs
DEV=2
. .. jones smith
DEV=3
48 . .. cos285 cos350
77
77 . 48 .. book 100 src
DEV=4
100 . 77 .. hw1 hw2 103 hw3
103 . 100 .. file1 file2 file3 subdir1

NAME

getcwd - Get current working directory

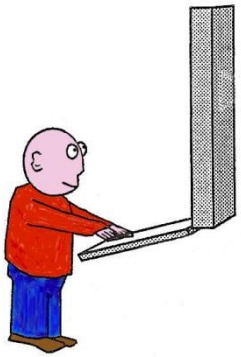
SYNOPSIS

```
#include <unistd.h>

char *getcwd(char *buf, size_t size);
```

DESCRIPTION

The getcwd() function copies an absolute pathname of the current work-
ing directory to the array pointed to by buf, which is of length size. The pathname is returned as the function result and via the argument buf.

If the length of the absolute pathname of the current working directory, including the terminating null byte, exceeds size bytes, NULL is returned, and errno is set to ERANGE; an application should check for this error, and allocate a larger buffer if necessary.



```
ln -s ../
parent
cd parent
cd l12
cd parent
cd l12
pwd
type pwd
/bin/pwd
```